Отчет по Творческой Работе

Самое начало:

Изучение датасета

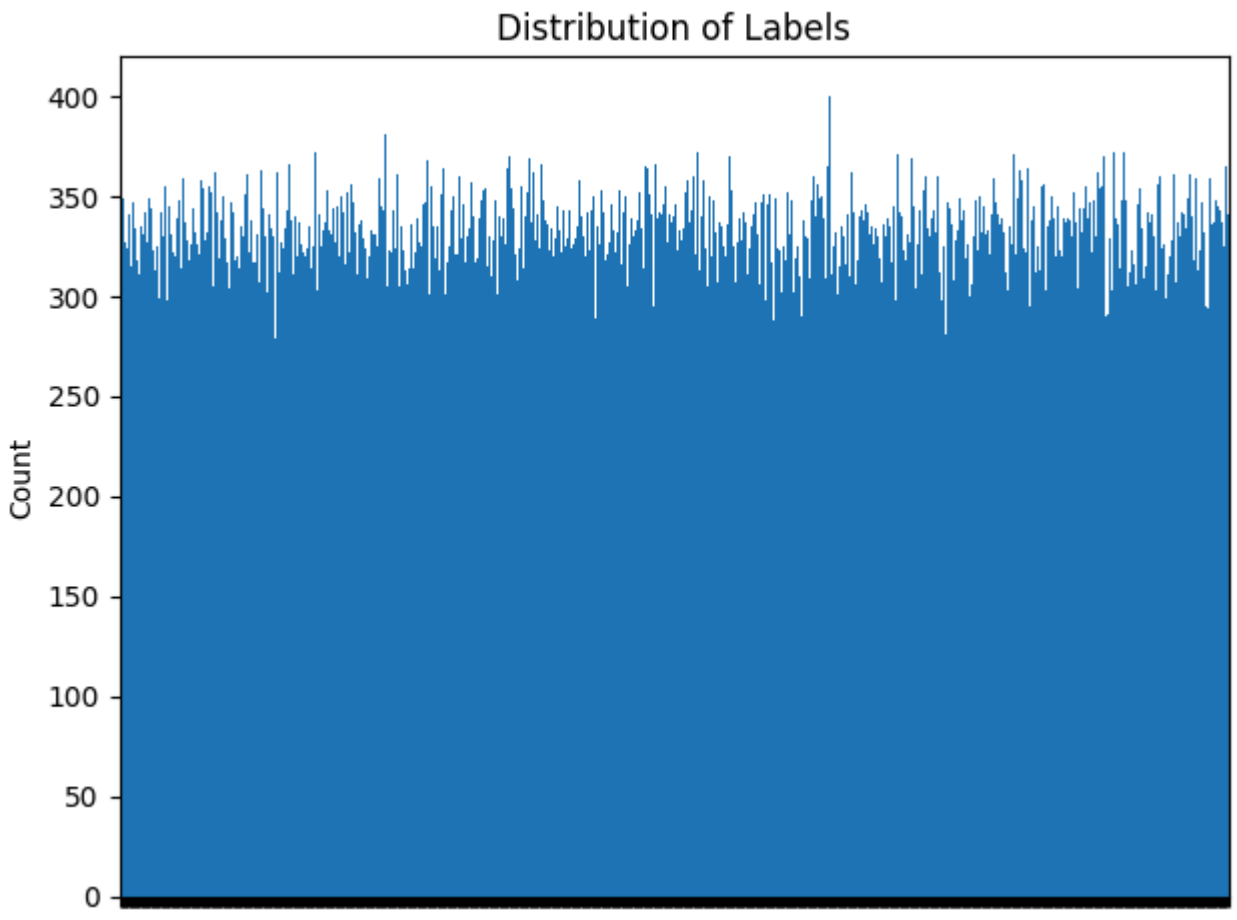
```
data = np.concatenate([data1, data2, data3, data4])
data.shape
(332987, 2)
```

Всего дано 332987 пары иероглиф - лейбл

```
labels = data[:, 1]
n_classes = np.unique(labels).shape[0]
n_classes
1000
```

Уникальный классов - 1000

Посмотрел распределение классов :
в целом равномерно распределено, совсем аномалий нет



Посмотрел как выглядят сами иероглифы



Посмотрел как выглядят иероглифы с одинаковым лейблом



ну или



Еще важный момент, картинки не одинакового размера

```
from collections import Counter
import pandas as pd

shapes = [data[i][0].shape for i in range(len(data))]
shape_counts = Counter(shapes)
shapes = list(shape_counts.keys())
df_shapes = pd.DataFrame(shapes, columns=['Height', 'Width'])
df_shapes.describe()
```

	Height	Width
count	8092.000000	8092.000000
mean	88.395823	77.680919
std	28.178315	26.171275
min	8.000000	13.000000
25%	66.000000	57.000000
50%	88.000000	77.000000
75%	109.000000	98.000000
max	207.000000	164.000000

Тут надо было принять решение к какому размеру их всех приводить, а они прям сильно раскиданы, решил что 64x64

После покопался с трансформациями и решил использовать что-то такое

```
transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

картинки получились такие:



очень даже неплохо

Бейзлайн

У меня было желание сделать некий бейзлайн на cnn и мой бейзлайн состоял из такого:

```
n_classes = 1000

class BasicBlockNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=5, padding=2)
        self.bn = nn.BatchNorm2d(32)
        self.relu = nn.ReLU()
```

```
        self.pool = nn.AvgPool2d(kernel_size=4)
        self.fc = nn.Linear(32 * (IMG_SIZE//4) * (IMG_SIZE//4), n_classes)  # flatten before FC
    def forward(self, x):
        x = self.conv(x)
        x = self.bn(x)
        x = self.relu(x)
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x
```

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9, weight_decay=1e-4)
train(
    model=model,
    optimizer=optimizer,
    train_loader=train_loader,
    val_loader=val_loader,
    device=device,
    n_epochs=10,
    criterion=F.cross_entropy
)
```

и оно прекрасно тренировалось, на трейне и вале следующие значения:

`Epoch 10/10`

`Train Loss: 0.0055, Train Acc: 0.9987`

`Val Loss: 0.5859, Val Acc: 0.9177`

только вот на тесте

`test loss: 6.9305, test accuracy: 0.0010`



удивительно, но test_accuracy совпадает с accuracy при предсказании константного класса

подумал, подумал и решил что не просто так нам дали resnet, что это некая база и попробовал использовать его втупую

```
n_classes = 1000
pool_size = 4
class BasicBlockNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.shortcut = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=1)
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

```
        self.bn1 = nn.BatchNorm2d(32)
        self.relu = nn.ReLU()
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=32, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)
        self.pool = nn.AvgPool2d(kernel_size=pool_size)
        self.fc = nn.Linear(32 * (IMG_SIZE//pool_size) * (IMG_SIZE//pool_size), n_classes)
    def forward(self, x):
        stub = self.shortcut(x)
        x = self.relu(self.bn1(self.conv1(x)))
        x = self.bn2(self.conv2(x))
        x += stub
        x = self.relu(x)
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)

        return x
```

```
train_loss_log, train_acc_log, val_loss_log, val_acc_log = train(net, optimizer, 10, train_loader, val_loader)
```

Прокнуло,
на train & val:
`Epoch 10/10` train loss: 0.011775638643333189, train acc: 0.9973322124622563
`val loss: 0.6448508787634287, val acc: 0.9151099780610216 на своем тесте` test loss: 0.6834, test accuracy: 0.9128
но на kaggle
`Score: 0.81487`

Изменяшки

0.. Начал я с шедулеров, в бейзлайне у меня не было никакого, попробовал MultiStepLR, но буквально ничего не поменялось, буквально точь-в-точь метрики 🤷

1.. Потом я поменял шедулер на CosineAnnealingLR
(`scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=10)`
и зашло прям хорошо, на 2% accuracy поднялось на тес
те кэгла
Было: (0.81848; 0.81487), Стало: (0.83876; 0.83433)
2.. Дальше пришла в голову поменять размер ядра пулинга
(я буду называть это poolsize, не знаю есть ли такой термин)
тк размер у нас 64x64 и в бейзлайне poolsize был 4, как следствие получалось много квадратиков!! и терялась некая обобщающая способность
сначала я поменял poolsize на 8 и неожиданно хорошо зашло
(0.83433; 0.83876) против (0.87746; 0.87561)

3.. Я решил если увеличение poolsize-а так зашло, то где увеличение poolsize-а 2???
хахахахха, и оно реально зашло
(0.87746; 0.87561) -> (0.89512; 0.89482), не 4% как в прошлый раз но тоже шик

4.. Я был наслышан про практику увеличивать количество слоев свертки ну и кажется самое время ее применить

```
def forward(self, x):
    stub = self.shortcut(x)

    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)

    x = self.conv2(x)
    x = self.bn2(x)
    x = self.relu(x)

    x = self.conv3(x)
    x = self.bn3(x)
```

```
        x = self.relu(x)

        x = self.conv4(x)
        x = self.bn4(x)
        x += stub
        x = self.relu(x)
        x = self.pool(x)

        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x
```

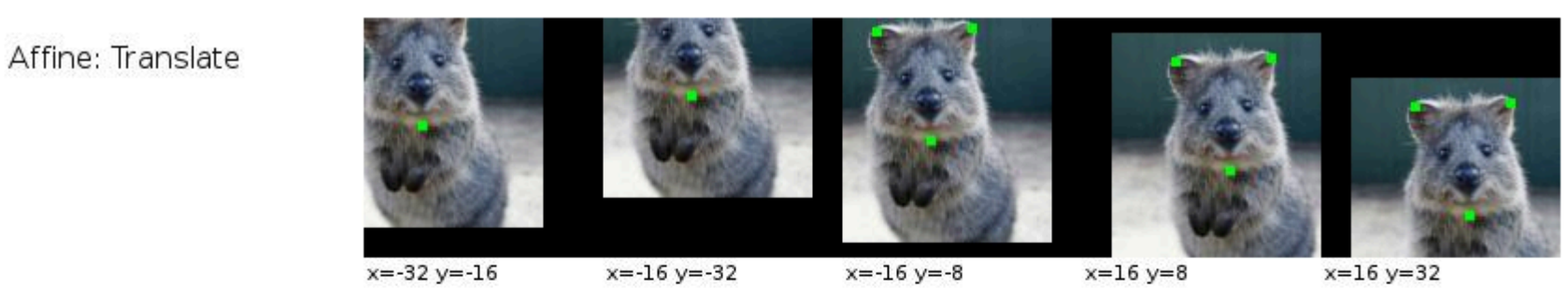
забабахал 4 слоя и это дало прирост в 3%
(0.89512; 0.89482) -> (0.92443; 0.92357), получилось достойно, только моделька стала еще дольше обучаться 💀

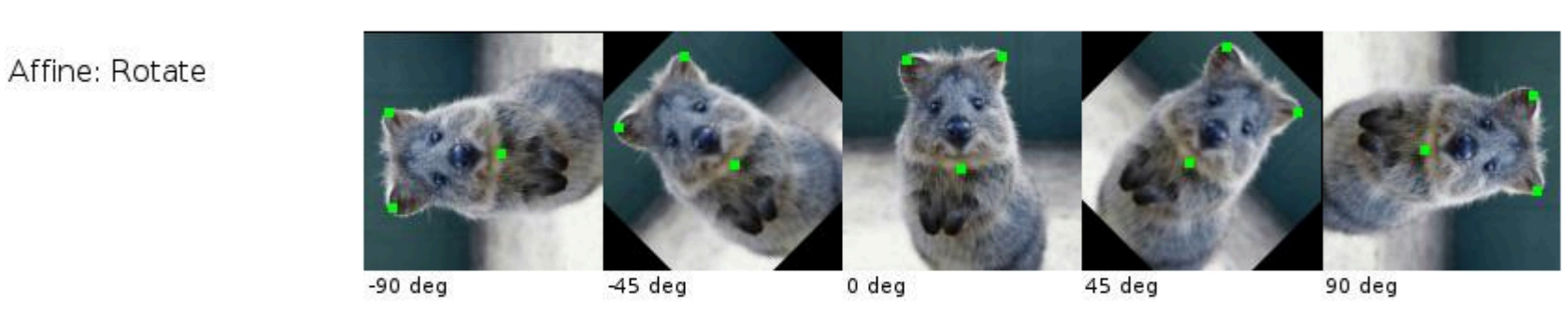
5.. короче мне понравилось то что просто увеличение кол-ва сверток дает прирост поэтому добавил пятый слой сверток и как-то захотелось сразу добавить аугментацию
напихал всего и остался доволен, только пришлось потом разгребать что дало улучшение а что нет

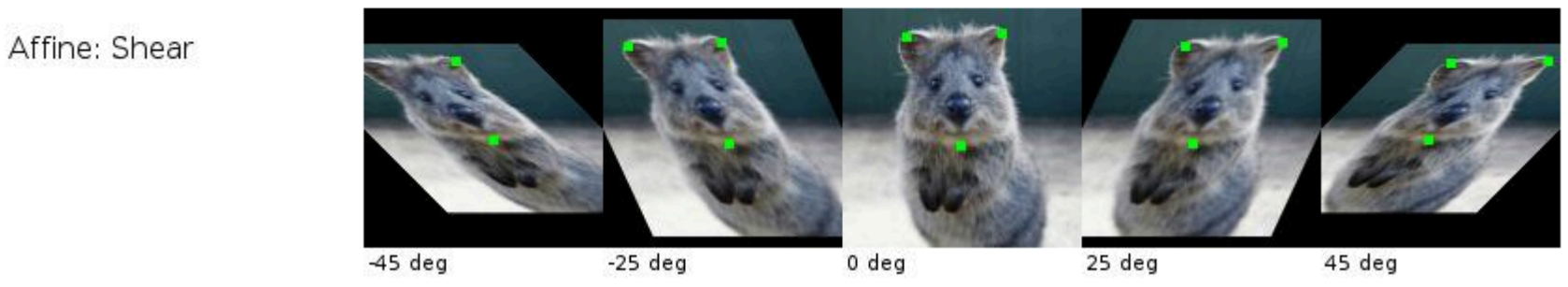
Как я выбирал аументации:
То что точно не подойдет:

- transforms.HorizontalFlip()
- transforms.VerticalFlip()
- transforms.RandomResizedCrop()
  Первые два дадут вообще полную ерунду, иероглиф станет чем-то другим, это как если перевернуть букву
  Третье обрежет иероглиф, а это НЕ то что бы хотелось
  То что казалось хорошей практикой:

1. transforms.RandomAffine(degrees=5, translate=(0.05, 0.05), scale=(0.95, 1.05), shear=5)
2. transforms.RandomRotation(5)
3. transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 0.5))
4. transforms.ColorJitter(brightness=0.2, contrast=0.2)
   первое применяет различные трансформации, стало понятно картинкой







   при небольших значениях казалось что пригодится, тк люди по-разному пишут иероглифы и с разным наклоном (можно посмотреть на иероглифы с одинаковым лейблом, в главе "Изучение датасета")

второе просто применяет поворот, классика, но полезно

третье, блюр, пригодится, так как видно что некоторые иероглифы более четкие чем другие

джиттер в теории прикольно, но не захотел четвертую аугментацию добавлять

что в итоге получилось:

```
transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomRotation(5),
    transforms.RandomAffine(degrees=5, translate=(0.05, 0.05), scale=(0.95, 1.05), shear=5),
    transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 0.5)),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

на этом этапе:
(0.92443; 0.92357) -> (0.92719; 0.93019)

6.. от аугментаций оставил только поворот
(0.92719; 0.93019) -> (0.92996; 0.92936)
7.. добавил блюр
(0.92996; 0.92936) -> (0.92695; 0.92907)
8.. убрал блюр, добавил афины (как это вообще переводится??)
(0.92695; 0.92907) -> (0.92251; 0.92284)

получилось так, что блюр мешался, афины мешались, а поворот хоть и не сильно но улучшает показания

9.. оставил только поворот, и добавил т.н. skip connections

получилась такая структура:

```
global_sc = self.globalshortcut(x)
sc1 = self.shortcut1(x)
x = self.conv1(x)
x = self.bn1(x)
x += sc1
x = self.relu(x)

sc2 = self.shortcut2(x)
x = self.conv2(x)
x = self.bn2(x)
x += sc2
x = self.relu(x)

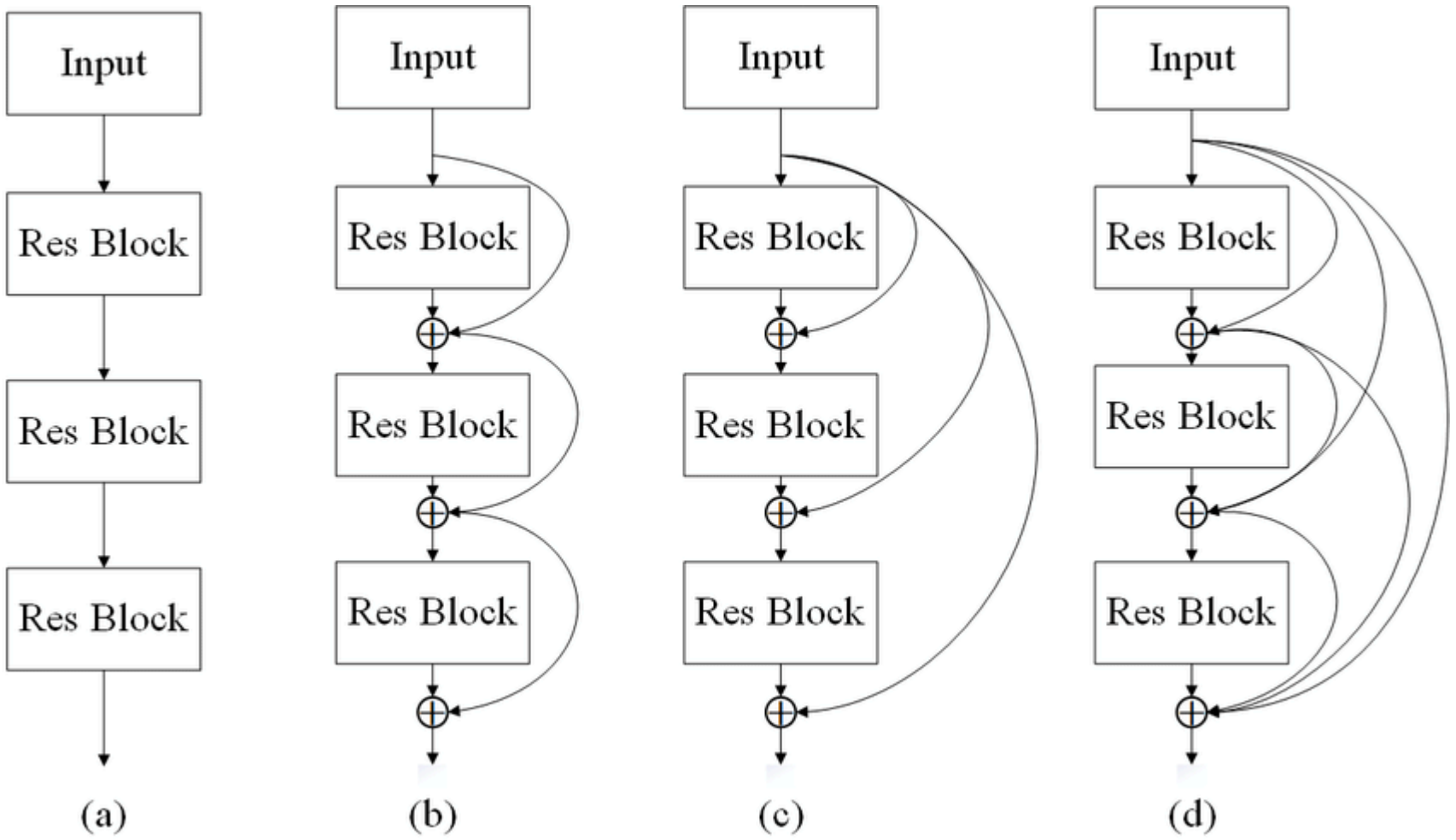
sc3 = self.shortcut3(x)
x = self.conv3(x)
x = self.bn3(x)
x += sc3
x = self.relu(x)

sc4 = self.shortcut4(x)
x = self.conv4(x)
x = self.bn4(x)
x += sc4
x = self.relu(x)

x = self.conv5(x)
x = self.bn5(x)
x += global_sc
x = self.relu(x)
x = self.pool(x)

x = x.view(x.size(0), -1)
x = self.fc(x)
return x
```

не уверен что это было правильно, делал как чувствовал и по схемам



обучалось это чудо 114 минут

(0.93116; 0.92959)
ну кажется неплохо, хотя бы не меньше!

10.. Поменял размер ядра в conv с 3 до 5 и это дало хороший результат
так как у нас большая картинка (64x64), поэтому размер ядра 3, кажется смена размера имеет смысл
(0.93116; 0.92959) -> (0.94533; 0.94248)
очень даже хороший результат!

Перспективы

Я до дедлайна не успею попробовать все методы, поэтому запишу их в перспективы

- Dilated Convolution, кажется хорошая практика и позволит увеличить область зрения модельки, а ключи в иероглифах большие и по ощущениям даст прирост к качеству модели
- Drop Out, тоже хочется попробовать, потому что и на лекциях говорили про него, и с переобучением поможет
- Более сложные аугментации может быть и другие шедулеры

Итоги

хочется написать что из бейзлайна (0.81487; 0.81848) получилась хорошая моделька (0.94533; 0.94248)
то есть прирост на 13%,
ну и мне понравилось обучать и пробовать разные методы и схемы